

Punto 5.6 — Migraciones en entornos de producción: estrategias y despliegue

Contexto del proyecto

Este punto continúa directamente M05/PROYECTO/5.5. La carpeta M05/PROYECTO/5.6 contiene la solución completa `AceriaData.sln` y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Separar desarrollo y despliegue controlado mediante `IMigrator`, scripts revisables y migration bundles, conservando el historial real.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.6
dotnet restore AceriaData.sln
```

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- `src/AceriaData.Infrastructure/Repositories/MigracionesProduccionM5Repositorio.cs`
- `deployment/generate-production-artifacts.ps1`
- `deployment/PLAN_DESPLIEGUE.md`

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar `MigracionesProduccionM5Repositorio.cs`

```
public async Task<MigracionesProduccionM5Dto> AplicarConIMigratorAsync()
{
    using var scope = _scopeFactory.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();

    var migrator = context.GetService<IMigrator>();
    await migrator.MigrateAsync();

    var aplicadas = (await context.Database.GetAppliedMigrationsAsync()).ToArray();
    var pendientes = (await context.Database.GetPendingMigrationsAsync()).ToArray();
    var tablaExiste = await ExisteTablaHistorialAsync(context);

    return new MigracionesProduccionM5Dto(
        aplicadas.Length,
        pendientes.Length,
        aplicadas.LastOrDefault() ?? "<ninguna>",
        tablaExiste,
        "__EFMigrationsHistory");
}
```

Explicación línea a línea — `MigracionesProduccionM5Repositorio.cs`

Línea 1: `public async Task<MigracionesProduccionM5Dto> AplicarConIMigratorAsync()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `using var scope = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 4: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infraestructura sea la misma que en la aplicación.

Línea 6: `var migrator = context.GetService<IMigrator>();` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 7: `await migrator.MigrateAsync();` → Aplica la cadena real de migraciones al proveedor de pruebas o despliegue.

Línea 9: `var aplicadas = (await context.Database.GetAppliedMigrationsAsync()).ToArray();` → Lee la lista de migraciones que la base registra como aplicadas.

Línea 10: `var pendientes = (await context.Database.GetPendingMigrationsAsync()).ToArray();` → Comprueba si queda alguna migración del ensamblado pendiente de aplicar.

Línea 11: `var tablaExiste = await ExisteTablaHistorialAsync(context);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 13: `return new MigracionesProduccionM5Dto(` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 14: `aplicadas.Length,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 15: `pendientes.Length,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 16: `aplicadas.LastOrDefault() ?? "<ninguna>",` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 17: `tablaExiste,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 18: `"__EFMigrationsHistory");` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 19: `}` → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar `generate-production-artifacts.ps1`

```
$ErrorActionPreference = "Stop"
$Artifacts = Join-Path $PSScriptRoot "artifacts"
New-Item -ItemType Directory -Force -Path $Artifacts | Out-Null

dotnet ef migrations script --idempotent `
  --project ../src/AceriaData.Infrastructure `
  --startup-project ../src/AceriaData.Console `
  --configuration Release `
  --output (Join-Path $Artifacts "aceria-idempotent.sql")

if ($LASTEXITCODE -ne 0) { throw "No se pudo generar el script idempotente" }

dotnet ef migrations bundle `
  --project ../src/AceriaData.Infrastructure `
  --startup-project ../src/AceriaData.Console `
  --configuration Release `
  --output (Join-Path $Artifacts "aceria-efbundle.exe") `
  --force

if ($LASTEXITCODE -ne 0) { throw "No se pudo generar el migration bundle" }
```

Explicación línea a línea — generate-production-artifacts.ps1

Línea 1: `$ErrorActionPreference = "Stop"` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 2: `$Artifacts = Join-Path $PSScriptRoot "artifacts"` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 3: `New-Item -ItemType Directory -Force -Path $Artifacts | Out-Null` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 5: ``dotnet ef migrations script --idempotent ``` → Genera SQL desde la cadena real de migraciones de EF Core.

Línea 6: ``--project ../src/AceriaData.Infrastructure ``` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 7: ``--startup-project ../src/AceriaData.Console ``` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 8: ``--configuration Release ``` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 9: `--output (Join-Path $Artifacts "aceria-idempotent.sql")` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 11: `if ($LASTEXITCODE -ne 0) { throw "No se pudo generar el script idempotente" }` → Convierte un fallo del comando externo en un fallo explícito del script.

Línea 13: ``dotnet ef migrations bundle ``` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 14: ``--project ../src/AceriaData.Infrastructure ``` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 15: ``--startup-project ../src/AceriaData.Console ``` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 16: ``--configuration Release ``` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 17: ``--output (Join-Path $Artifacts "aceria-efbundle.exe") ``` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 18: `--force` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 20: `if ($LASTEXITCODE -ne 0) { throw "No se pudo generar el migration bundle" }` → Convierte un fallo del comando externo en un fallo explícito del script.

Paso 5: Implementar y estudiar Comandos de despliegue

```
dotnet ef migrations script --idempotent `
  --project src/AceriaData.Infrastructure `
  --startup-project src/AceriaData.Console `
  --configuration Release `
  --output aceria-idempotent.sql

# El bundle recibe la conexión en ejecución, no desde código fuente.
./aceria-efbundle.exe --connection $env:ACERIA_PROD_CONNECTION
```

Explicación línea a línea — Comandos de despliegue

Línea 1: ``dotnet ef migrations script --idempotent ``` → Genera SQL desde la cadena real de migraciones de EF Core.

Línea 2: `--project src/AceriaData.Infrastructure` `` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 3: `--startup-project src/AceriaData.Console` `` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 4: `--configuration Release` `` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 5: `--output aceria-idempotent.sql` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 7: `# El bundle recibe la conexión en ejecución, no desde código fuente.` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 8: `./aceria-efbundle.exe --connection $env:ACERIA_PROD_CONNECTION` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Paso 6: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

La ejecución debe finalizar con `5.6 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- MigrationsAssembly apunta al ensamblado real de migraciones.
- IMigrator aplica la cadena y deja cero migraciones pendientes.
- Se conserva `__EFMigrationsHistory`.
- El bundle recibe la cadena de conexión desde el entorno de despliegue.
- El plan distingue downgrade de recuperación de datos.

Reto resuelto y ampliación

Reto: Genera un script revisable y un bundle para la misma cadena y documenta qué control operacional aporta cada uno.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Ejecutar Database.Migrate en cada réplica como regla general de producción.
- Renombrar MigrationsHistoryTable sin mover el historial existente.
- Guardar credenciales de producción en el repositorio.
- Confundir un Down destructivo con restauración segura.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.6 OK`.

3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.6, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.6 OK

Conexión con el siguiente punto

El siguiente estado es 5.7 y parte directamente de este proyecto completo.

Reconstrucción editorial M5: los puntos 5.1–5.6 conservan la versión corregida y ya cerrada técnicamente. Desde 5.7 se reconstruye la práctica desde las fuentes originales del usuario y se aplican únicamente las correcciones técnicas explícitas del contrato M5.